OpenID Connect and SSO

OAuth was never built for login

Watch the 5-min video

In one line: OpenID Connect adds a signed **ID token** to the OAuth 2.0 code flow, saying who logged in, when, and for which app; with discovery and one trusted identity provider, many apps get single sign-on.

1 OAuth isn't login

A developer logs users in with plain OAuth: the app gets an access token and assumes whoever holds it is the user. But an access token says what the bearer may *access*, not *who* they are, and it may have been issued for a different app (a malicious app can replay a token it got for itself). OAuth 2.0 (lesson 42) was built for delegation. **OpenID Connect** (OIDC Core 1.0, 2014) was built for login.

2 An identity layer on OAuth 2.0

The client adds the `openid` scope to a normal authorization code flow (with PKCE). The token response now includes an `id_token`: a signed JWT (lesson 41) issued **for this client**. The access token is still there, but only for calling APIs.

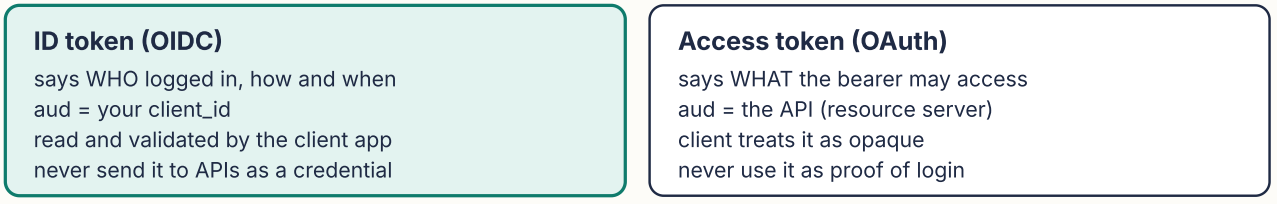


Figure 1. Two tokens, two audiences. Mixing them up is the classic OIDC mistake.

Validate the ID token: signature (keys from the provider's JWKS), `iss` = the expected issuer, `aud` contains your `client_id`, `exp` not passed, and `nonce` equals the value you sent at the start (stops replay). Identify the user by `iss` + `sub`, not by email: emails change and can be reassigned. Extra profile claims (`name`, `email`, `email_verified`) come in the token or from the **userinfo** endpoint.

3 Discovery

Every provider publishes `<issuer>/.well-known/openid-configuration`, a JSON document listing `authorization_endpoint`, `token_endpoint`, `userinfo_endpoint`, `jwks_uri` and supported scopes. A library needs only the issuer URL to configure itself, so moving from Google to Okta or Entra ID is mostly configuration.

4 Single sign-on

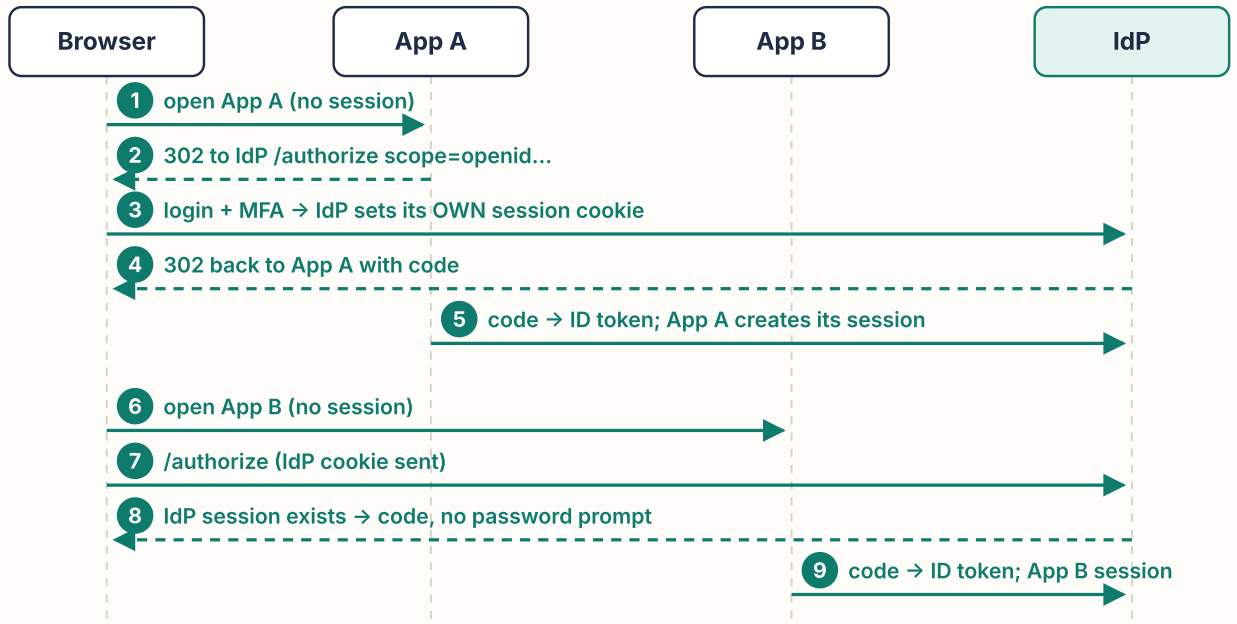


Figure 2. SSO: the identity provider keeps its own session. The second app redirects there, and the user comes straight back with a fresh ID token.

Many apps trust one identity provider. One password, one place for MFA and conditional access, and one place to disable a leaving employee. Each app still keeps its **own** session after login. Single *logout* is harder: OIDC has RP-initiated, front-channel and back-channel logout specs, but each app must implement them.

	SAML 2.0	OpenID Connect
Format	XML assertions, XML signatures	JSON, JWT (ID token)
Built on	browser redirects and POSTs	OAuth 2.0 authorization code flow
Good for	existing enterprise web apps	SPAs, mobile apps, APIs, new systems
Also gives API access?	no (separate mechanism)	yes, the access token from the same flow

WORKED EXAMPLE: ENTERPRISE SIGN-IN WITH ENTRA ID

The app is registered in Entra ID (formerly Azure AD) with its redirect URI. Login uses the code flow with PKCE and `scope=openid profile email`. The app validates the ID token: signature, `iss = https://login.microsoftonline.com/<tenant-id>/v2.0`, `aud` = its client ID, `nonce`, expiry. It keys the user on `tid` + `oid` (stable across the tenant's apps) and creates its own session cookie. Employees sign in once and reach every company app.

COMMON MISTAKES

- Using the access token, or a userinfo call alone, as proof of login.
- Accepting an ID token whose `aud` is another app's client ID.
- Skipping the `nonce` check, or matching users by email instead of `iss` + `sub`.
- Sending the ID token to APIs as a bearer token.

5 Key terms

OIDC: identity layer on OAuth 2.0 for login

nonce: random value tying the ID token to this login request

userinfo endpoint: returns profile claims for an access token

ID token: signed JWT about the login, aud = the client

Discovery document: /.well-known/openid-configuration

SSO: one IdP session signs the user into many apps

6 Interview questions

Q1. What does OIDC add on top of OAuth 2.0? BASIC

A standard, signed ID token that says who logged in and for which app (iss, sub, aud, nonce), plus the userinfo endpoint and discovery documents.

Q2. What is single sign-on? BASIC

Many apps trust one identity provider; the user logs in once there and is signed into every app without re-entering credentials.

Q3. Your company has 30 internal apps with separate logins. How do you consolidate? SCENARIO

Register each app with one IdP (Entra ID, Okta) using OIDC, or SAML for legacy apps; enforce MFA there, and disable accounts in one place.

Q4. Which ID token checks must your app perform? SCENARIO

Signature against the JWKS, issuer, audience equal to your client ID, expiry, and the nonce you sent.

Q5. Why is an OAuth access token not proof of identity for your app? TRICKY

It was issued for an API, may be opaque, and could have been obtained by another app and replayed to you; nothing binds it to your client.

Q6. Why key users on iss + sub instead of email? TRICKY

sub is the provider's stable, unique ID; emails change, can be reused, and may be unverified at some providers, which enables account takeover.

Next: SD-M-006 · RBAC and ABAC backendroadmap.com